Repository Creation and Maintenance (GitHub)

Student: Dharshan Amin
Repository: daworld10/Project-management
Repository Link: https://github.com/daworld10/Project-management
Project Theme: Integrating Waterfall and Agile Methodologies in Data Analysis (Credit Score Classification Case Study)

1. Repository Purpose and Scope

This repository was created to serve as the single source of truth for the Project Management coursework deliverables connected to the credit score classification case study. The scope includes documentation artifacts (final paper, standups, process maps), presentation material, and supporting references needed to demonstrate methodology integration and project execution evidence.

The repository is organized as a course deliverables repository rather than a production software repository; therefore, emphasis is placed on maintainable documentation, traceable updates, and easy navigation for reviewers.

2. Repository Creation

The repository was created to centralize all project artifacts such as research paper drafts, diagrams (Waterfall/Wardley), presentation files, weekly standup evidence, and supporting documentation, so everything remained accessible and version controlled throughout the semester. Establishing a clear repository structure early made it easier to track progress, share deliverables, and avoid losing files across multiple local devices.

- **Naming and purpose clarity:** The repository name and description were kept aligned with the project topic, so it is immediately clear what the repo contains and why it exists.
- **README documentation:** A README.md file was included to explain what the project is, what files exist, and how to navigate the repository.
- **Standardized structure:** Files were added in a consistent way so that final deliverables (paper/presentation) and supporting evidence (boards/standups/diagrams) are easy to locate.

3. Repository Maintenance

Ongoing maintenance focused on keeping the repository organized, traceable, and easy to audit, which is essential when the repo is used as both a submission artifact and a project-management record. Maintenance activities were treated as part of the workflow rather than an afterthought, since documentation and traceability are key expectations in a project management course.

- **Commit history discipline:** Changes were committed with descriptive messages and kept as small, focused updates ("atomic commits") so the evolution of the project remains understandable.
- **Pull request and review practice (when used):** Reviews ensure changes are correct and discussions are resolved before merging, improving quality and reducing mistakes.
- **Issue/Task organization:** Issues (or task items) can be labelled and grouped using milestones to reflect priority and align work with sprint goals.

4. Version Control Workflow Insights

Using GitHub throughout the project made the work easier to manage because it acted as both a version-control system and a lightweight coordination space. Instead of keeping files scattered across devices or repeatedly renaming "final_v2/final_v3," the repository provided a reliable history of changes and a single source of truth for the latest deliverables.

- **Better traceability and accountability:** Each update is recorded through commits, so it is easy to understand what changed, when it changed, and why it changed based on commit messages. This also reduces confusion when revisiting the project after a gap, because progress is visible in the history rather than dependent on memory.
- **Clearer coordination:** GitHub supports structured collaboration through issues, comments, and pull requests, which helps keep discussions connected to the exact task or file being worked on. Even in small teams (or solo work with peer feedback), this reduces back-and-forth and keeps decisions documented.
- **Planning support without extra tools:** Project boards, labels, and milestones allow tasks to be organized in a way that fits Agile workflows (backlog → in progress → done) without needing a separate project-management platform. This helped keep implementation work and task tracking in the same environment.

- **Improved habits and learning:** Regular use of commits, branching (when needed), and organized documentation builds practical workflow discipline that is useful beyond this course. Exploring optional tooling such as AI-assisted coding features can also speed up routine work and encourage experimentation, as long as outputs are still reviewed critically.

Overall, GitHub increased productivity by combining file management, progress tracking, and documentation in one place, which made the project easier to maintain and easier to present as evidence of structured work.

5. Task and Workflow Management in GitHub

GitHub was used for more than storing files, it also supported day-to-day project coordination by linking tasks, discussions, and deliverables in one place. This reduced tool-switching and made it easier to show progress through an auditable trail of issues, commits, and board updates.

- **Visual workflow tracking:** GitHub project boards provide a Kanban-style view of work, helping move items through clear stages (for example: planned → active → completed). Filters and labels make it easier to focus on priority items and quickly see what is blocked or pending review.
- **Task structuring with metadata:** Issues can be organized using assignees, labels, and milestones so responsibilities and timelines stay clear. Milestones help connect tasks to sprint goals or weekly deliverables rather than keeping work as an unstructured to-do list.
- **Collaboration tied to artifacts:** Comments, mentions, and reviews keep communication attached to the exact task or file being discussed, which improves clarity and reduces repeated explanations. Pull request reviews (when used) also create a simple quality checkpoint before changes become part of the main version.
- **Supports Agile execution:** GitHub fits Agile-style working (Kanban or sprint-based) because tasks can be reprioritized quickly, progress is visible continuously, and milestone/sprint planning stays connected to actual commits and deliverables.